

O'REILLY®

GCP Professional ML Engineer: Crash Course

Instructor: Richard Bryant





Learning Objectives

By the end of this course, you will understand:

- Performing data processing and ML in GCP using the appropriate tool for the job
- Performing feature engineering and feature crosses at the right time
- Instantly recognize problems such as the bias-variance trade-off and data leakage when training and serving models
- Serving models using Google Cloud appropriately using batch or online prediction
- Apply an appropriate pipeline orchestration framework for a problem





GCP Professional ML Exam

I highly recommend starting with Google's certification exam guide:

<https://cloud.google.com/learn/certification/guides/machine-learning-engineer>

- Section 1: Architecting low-code ML solutions (~12%)
- Section 2: Collaborating within and across teams to manage data and models (~16%)
- Section 3: Scaling prototypes into ML models (~18%)
- Section 4: Serving and scaling models (~19%)
- Section 5: Automating and orchestrating ML pipelines (~21%)
- Section 6: Monitoring ML solutions (~14%)

The exam is 60 multiple-choice questions in 120 minutes.



About this course

- We will condense all the categories covered in the exam into a few sections
- 30 representative questions of varying difficulties will be featured and reviewed
- Sources:
 - “Official Google Cloud Certified Professional Machine Learning Engineer Study Guide” by Mona Mona and Pratay Ramamurthy
 - “Journey to Become a Google Cloud Machine Learning Engineer” by Logan Song
 - Google Cloud’s official sample questions and documentation



Fundamentals





When use ML?

- Many, but not all business cases are appropriate for ML
 - Need to understand if there are existing tools that are better than ML
 - What's the intended impact? What does success look like? What's the budget and time frame?
 - Is the problem even solvable using ML?



Approaches

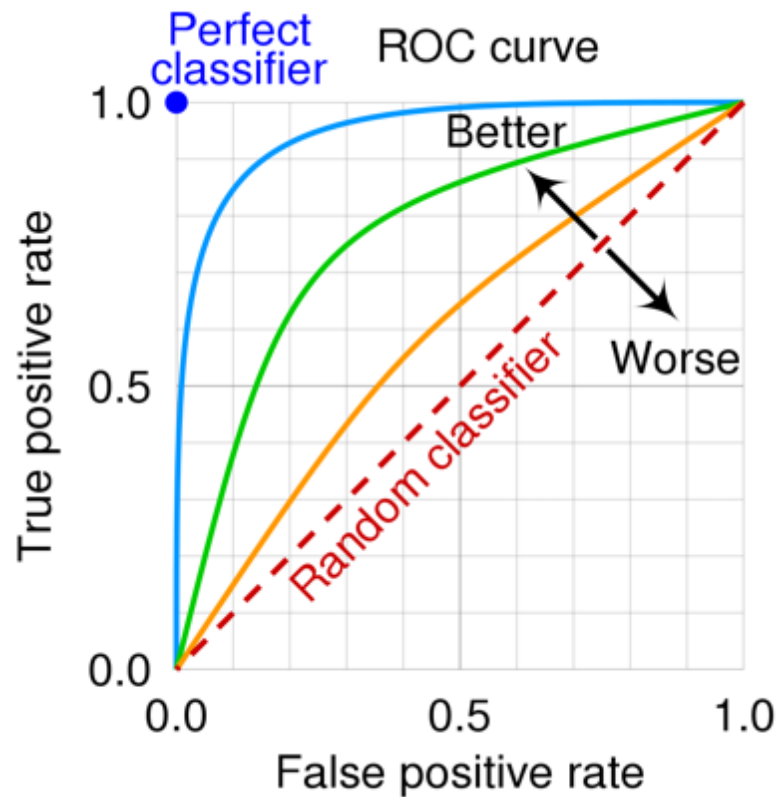
- **Supervised Learning** – when we have a labeled dataset that we can use to train a model
 - *Classification* – predicting labels or categories
 - *Regression* – predicting a continuous response
 - *Forecasting* – have time-series data and predict the future given the past
- **Unsupervised Learning** – we have no labels and group them ourselves
 - *Clustering* – create groups based on similar within and difference between
 - *Topic Modeling* – take documents and classify based on commonality of words and/or sentences

Classification Metrics

- Metrics for accuracy of a classification model depend on use-cases. Some common ones are as follows:
 - Recall = (True Positive) / (True Positive + False Negative)
 - Precision = (True Positive) / (True Positive + False Positive)
- **Area Under the Precision Recall Curve (AUC PR):** Shows the relationship between precision and recall for different classification thresholds; useful for highly imbalanced datasets
- **Area Under the Receiver Operating Characteristic Curve (AUC ROC):** Shows the relationship between the false positive and true positive rate
 - Scale invariant & threshold invariant

		Predicted condition	
		Predicted Positive (PP)	Predicted Negative (PN)
Actual condition	Total population = P + N		
	Positive (P) ^[a]	True positive (TP), hit ^[b]	False negative (FN), miss, underestimation
	Negative (N) ^[d]	False positive (FP), false alarm, overestimation	True negative (TN), correct rejection ^[e]

ROC curve





Imbalanced Data

- We say we have imbalanced data when we have vastly unequal samples of the different classes in a dataset
 - Example: A spam email filter where, out of all emails, 1,000 are not spam and only 10 are spam
 - In a random sampling, you can:
 - **Oversampling:** duplicating cases from the minority class
 - **Undersampling:** removing cases from the majority class



Responsible AI

1. Be socially beneficial.

2. Avoid creating or reinforcing unfair bias.

3. Be built and tested for safety.

4. Be accountable to people.

5. Incorporate privacy design principles.

6. Uphold high standards of scientific excellence.

7. Be made available for uses that accord with these principles.

Reference: ai.google.com/responsibility/principles



Question #1

- You work for a car manufacturer and are asked to detect hidden cracks in engines using X-ray images. Missing a crack could mean the engine could fail at some random time while someone is driving the car. Cracks are relatively rare and happen in about 1 in 100 engines. You are going to build a machine learning model to classify if an engine has a crack or not. If a crack is detected, the engine would go through further testing to verify. What metric would you choose for your classification model?
 - A. Accuracy
 - B. Precision
 - C. Recall
 - D. RMSE



Question #1

- You work for a car manufacturer and are asked to detect hidden cracks in engines using X-ray images. Missing a crack could mean the engine could fail at some random time while someone is driving the car. Cracks are relatively rare and happen in about 1 in 100 engines. You are going to build a machine learning model to classify if an engine has a crack or not. If a crack is detected, the engine would go through further testing to verify. What metric would you choose for your classification model?
 - A. Accuracy
 - B. Precision
 - C. Recall
 - D. RMSE

RMSE is a regression metric. Accuracy will not be useful in situations of high imbalance. Because we don't want to miss any cracks, we want to maximize recall.



Question #2

- The data science team has built a DNN model to monitor and detect defective products using the images from the assembly line of an automobile manufacturing company. As a Google ML Engineer, you need to measure the performance of the ML model for the test dataset/images. Which of the following would you choose?
 - A. AUC
 - B. Precision
 - C. Recall
 - D. True positive rate



Question #2

- The data science team has built a DNN model to monitor and detect defective products using the images from the assembly line of an automobile manufacturing company. As a Google ML Engineer, you need to measure the performance of the ML model for the test dataset/images. Which of the following would you choose?
 - A. AUC
 - B. Precision
 - C. Recall
 - D. True positive rate

AUC is classification threshold invariant and is the best way to measure the model's overall performance.



GCP Tools





BigQuery



BigQuery is one of the most important tools in the GCP suite. It has the following features:

- Fully managed – Don't have to worry about issues like compute, storage, or other resources
- Scalable – BigQuery can handle petabytes of data
- Serverless
- Integrates seamlessly with Google Cloud Platform, Vertex AI, etc.
- Optimized for fast data processing
- Highly cost effective, doesn't bill for unused resources



BigQuery ML



You can use standard SQL queries in order to train ML models directly in BigQuery.

- Can export models directly to Vertex AI
- Useful for users who only know SQL who don't have advanced coding knowledge
- Variety of models possible, including: linear or logistic regression, time-series models, K-means clustering
- Can evaluate the models directly in BigQuery ML using `ML.EVALUATE()`
- See: https://cloud.google.com/bigquery-ml/docs/introduction#supported_models_in.



Vertex AI

This is another one of the most critical tools in GCP and is a significant foundation of the exam. It is a ML platform for building, deploying, and managing machine learning models at scale. Some of the key features:

- Vertex AI datasets
- Vertex AI Feature Store
- Vertex AI Vizier
- Vertex AI Workbench and notebooks
- Vertex AI model training
- Vertex AI Pipelines
- AutoML



Data ingestion

There are multiple GCP services for ingesting batch or streaming data.

- **Pub/Sub:** Short for Publish/Subscribe, provides asynchronous communication between applications and services.
 - Serverless, scalable service that integrates with services like Dataflow or BigQuery
 - Ideal for streaming data in real-time
- **Datastream:** Supports streaming from Oracle and MySQL databases into Cloud Storage
- **BigQuery Data Transfer Service:** Can load data from data warehouses (e.g. Teradata), Cloud Storage, or other cloud platforms



Dataproc



Dataproc is a managed Apache Spark and Apache Hadoop service that allows the user to run big data processing and analytics workloads.

- It integrates with other services like BigQuery, Cloud Storage, Data Studio, etc.
- Commonly used for ETL (Extract, Transform, Load) processes, batch processing, data exploration, and running analytics jobs on large datasets
- Automates the provisioning, scaling, and management of Spark and Hadoop clusters
- Can process structured data or unstructured data like videos, audio, images, etc.
- Specifically well-suited for batch processing



Dataflow



Dataflow is another fully managed service for building and executing data processing pipelines, for processing unstructured or structured data

- Uses a unified programming model based on Apache Beam
- Optimizes performance and cost efficiency by automatically scaling resources based on the workload
- Ideal for processing streaming data in real time, though it is designed for both batch and stream processing
- Focus on managed and serverless execution



Dataprep



Dataprep is a serverless and scalable data preparation service designed to handle data of various sizes.

- Enables users to visually explore, clean, and prepare data without the need for coding
- Suggests transformations and cleans data based on detected patterns
- No infrastructure to deploy or manage, is UI-based



Cloud Data Fusion

Cloud Data Fusion is a fully managed data integration service that enables users to build and manage ETL or ELT processes in a UI without coding.

- Offers a drag-and-drop interface for designing data pipelines
- No infrastructure management
- Allows MapReduce and Spark streaming pipelines and several connectors for dataases, storage systems, and SaaS applications
- Automatically captures metadata from data pipelines



NoSQL Data Store

NoSQL data options are optimized for single lookup operations (e.g. a single row with many columns). Latency of the following options can be in milliseconds:

- **Bigtable:** Fully managed NoSQL database service designed for massive scalability and low-latency storage and retrieval of structured data
 - Useful for real-time analytics e.g. fraud detection/ad prediction
- **Datastore:** A scalable, serverless NoSQL document database service designed for automatic scaling
 - Useful for web and mobile applications, metadata storage, user profiles, etc.
- **Memorystore:** A managed in-memory database, offering high-performance caching, session storage, and data caching; used for caching frequently accessed data



Pre-trained Models

GCP offers many pretrained models, and this should usually be the first go-to for model building, followed by AutoML, followed by custom models.

- Do not need coding or engineering experience to use pretrained models; we use APIs
- Serverless – we do not provision any cloud resources, only consume through an API
- Several pretrained options:
 - Vision AI, Video AI, Natural Language AI, Translation AI, Speech-to-Text, Text-to-Speech, Document AI



Pre-trained Model Examples

- **Vision AI:** Perform image classification, detect objects and faces
- **Video AI:** Recognize objects, places, and actions in videos
- **Natural Language AI:** Generates insights from unstructured text using pretrained ML models
- **Translation AI:** Detects more than 100 languages and translates between any pairs
- **Speech-to-Text:** Converts recorded audio or streaming audio into text
- **Text-to-Speech:** Converts text to speech with humanlike tone
- **Document AI:** Extracts info from documents (e.g. detect quality, extract key/value pairs, split and classify documents)



AutoML



VertexAI AutoML is a managed Google Cloud service that allows users to create ML models without writing any code.

- Ideal for users with little AI/ML or coding experience or with very quick turnaround requirements
- Supports: structured data, image data, text data, and video data
- Datasets are managed by Vertex AI
- Enables splitting between training, validation, and testing datasets, hyperparameter optimization, model selection, model evaluation, etc.



Question #3

- Your company deals with real estate, and as part of a software development team, you have been asked to add a machine learning model to identify objects in photos uploaded to your website. How do you go about this?
 - A. Use a custom model to get best results
 - B. Use AutoML to create object detection.
 - C. Start with Vision AI, and if that does not work, use AutoML.
 - D. Combine AutoML and a custom model to get better results.



Question #3

- Your company deals with real estate, and as part of a software development team, you have been asked to add a machine learning model to identify objects in photos uploaded to your website. How do you go about this?
 - A. Use a custom model to get best results
 - B. Use AutoML to create object detection.
 - C. Start with Vision AI, and if that does not work, use AutoML.
 - D. Combine AutoML and a custom model to get better results.

We generally want to start by trying pre-trained models. AutoML would be the next easiest solution; custom models should only be used if these are not options.



Question #4

- The data science team is building a data pipeline for an auto manufacturing company, whose objective is to integrate all the data sources that exist in their on-premise facilities, via a codeless data ETL interface. What GCP service will you use?
 - A. Dataproc
 - B. Dataflow
 - C. Dataprep
 - D. Data Fusion



Question #4

- The data science team is building a data pipeline for an auto manufacturing company, whose objective is to integrate all the data sources that exist in their on-premise facilities, via a codeless data ETL interface. What GCP service will you use?
 - A. Dataproc
 - B. Dataflow
 - C. Dataprep
 - D. Data Fusion

The key here is data integration with a codeless interface. Data Fusion is codeless and is specifically used for ETL pipelines.



Question #5

- You are a data scientist working on a smart city project to build an ML model to detect anomalies in real-time sensor data. You will use Pub/Sub to handle incoming requests. You want to store the results for analytics and visualization. How should you configure the below pipeline:
- Ingest data using Pub/Sub -> 1. Preprocess -> 2. ML training -> 3. Storage -> Visualization in Data Studio
 - A. 1. Dataflow, 2. Vertex AI Training, 3. BigQuery
 - B. 1. Dataflow, 2. Vertex AI AutoML, 3. Bigtable
 - C. 1. BigQuery, 2. Vertex AI Platform, 3. Cloud Storage
 - D. 1. Dataflow, 2. Vertex AI AutoML, 3. Cloud Storage



Question #5

- You are a data scientist working on a smart city project to build an ML model to detect anomalies in real-time sensor data. You will use Pub/Sub to handle incoming requests. You want to store the results for analytics and visualization. How should you configure the below pipeline:
- Ingest data using Pub/Sub -> 1. Preprocess -> 2. ML training -> 3. Storage -> Visualization in Data Studio
 - A. 1. Dataflow, 2. Vertex AI Training, 3. BigQuery
 - B. 1. Dataflow, 2. Vertex AI AutoML, 3. Bigtable
 - C. 1. BigQuery, 2. Vertex AI Platform, 3. Cloud Storage
 - D. 1. Dataflow, 2. Vertex AI AutoML, 3. Cloud Storage

Dataflow is the go-to for data processing. BigQuery is useful for storage of ML datasets going into visualization in Looker/Data Studio.



Question #6

- You are a data scientist working on a fraud detection model. You will use Pub/Sub to handle incoming requests. You want to store the results for analytics and visualization. How should you configure the following pipeline:
- 1. Ingest data -> 2. Preprocess -> 3. ML training and visualize in Data/Looker Studio
 - A. 1. Dataflow, 2. Vertex AI Training, 3. BigQuery
 - B. 1. Pub/Sub, 2. Dataflow, 3. BigQuery ML
 - C. 1. Pub/Sub, 2. Dataflow, 3. Vertex AI Training
 - D. 1. Dataflow, 2. Vertex AI AutoML, 3. Cloud Storage



Question #6

- You are a data scientist working on a fraud detection model. You will use Pub/Sub to handle incoming requests. You want to store the results for analytics and visualization. How should you configure the following pipeline:
- 1. Ingest data -> 2. Preprocess -> 3. ML training and visualize in Data/Looker Studio
 - A. 1. Dataflow, 2. Vertex AI Training, 3. BigQuery
 - B. 1. Pub/Sub, 2. Dataflow, 3. BigQuery ML
 - C. 1. Pub/Sub, 2. Dataflow, 3. Vertex AI Training
 - D. 1. Dataflow, 2. Vertex AI AutoML, 3. Cloud Storage

Pub/Sub is for ingestion of data while Dataflow is for preprocessing. BigQuery ML can perform ML training and storage before we visualize in Data/Looker Studio.



Question #7

- You are training a TensorFlow model on a structured dataset with 100 billion records stored in several CSV files. You need to improve the input/output execution performance. What should you do?
 - A. Load the data into BigQuery, and read the data from BigQuery.
 - B. Load the data into Cloud Bigtable, and read the data from Bigtable.
 - C. Convert the CSV files into shards of TFRecords, and store the data in Cloud Storage.
 - D. Convert the CSV files into shards of TFRecords, and store the data in the Hadoop Distributed File System (HDFS).



Question #7

- You are training a TensorFlow model on a structured dataset with 100 billion records stored in several CSV files. You need to improve the input/output execution performance. What should you do?
 - A. Load the data into BigQuery, and read the data from BigQuery.
 - B. Load the data into Cloud Bigtable, and read the data from Bigtable.
 - C. Convert the CSV files into shards of TFRecords, and store the data in Cloud Storage.
 - D. Convert the CSV files into shards of TFRecords, and store the data in the Hadoop Distributed File System (HDFS).

The key here is the enormous number of records. Bigtable is designed for the low latency reading of such a large number of records.



Q & A





Feature Engineering





What is feature engineering?

Feature engineering is the process of determining which features might be useful in training a model, and creating those features by transforming raw data.

Google's crash course: <https://developers.google.com/machine-learning/data-prep/transform/introduction>

- We determine which features might be useful in training a model
- Then we convert raw data from the dataset into different versions of those features
- Models need vector or numerical representations of features to perform mathematical computation



Transforming numeric data

- Two common types of transformations for numeric data:
 - Normalizing – transforming numeric data to the same scale as other numeric data
 - If numeric features have wildly different ranges, it can affect performance and can slow down convergence
 - Bucketizing – transforming numeric data to categorical data
 - One option: can create buckets with equal-spaced boundaries
 - You can also create buckets with quantile boundaries

Transforming categorical data

- **One-hot encoding** common for categorical data (Example from https://gist.github.com/ZhenghaoXiao/32/7d531c094984621b7fd3bca5b14f6cd3#file-one_hot_encoding_example-csv)
- Feature hashing applies a hash function to categorical features
 - Embedding represents a categorical feature as continuous-valued – often used for text/document classification

Search this file...				
1		color_red	color_green	color_blue
2	red	1	0	0
3	green	0	1	0
4	blue	0	0	1

one_hot_encoding_example.csv hosted with ❤ by GitHub [view raw](#)



Feature selection

- Feature selection refers to selecting a subset of features – reducing the number of input variables that are most useful to a model in order to predict a target
- Dimensionality reduction helps to:
 - Reduce noise from data and chance of overfitting
 - Less training time and use of computational resources
 - Increases performance of many machine learning algorithms
- To do this, we might try to select only the most important features, or create combinations of new features



Feature Crosses

A feature cross is a synthetic feature formed by “crossing” categorical or bucketed features.

- Suppose you are predicting housing prices for cities in California. We want to predict house sale prices based on the house location and rooms per person.
- House location measured with two features: [binned latitude] and [binned longitude]
- We could create a feature cross like: [binned latitude X binned longitude X rooms per person]
 - The model could possibly learn city-specific, neighborhood-specific, or even block-specific effects

Reference: <https://developers.google.com/machine-learning/crash-course/feature-crosses/check-your-understanding>



TensorFlow Data API

- **TensorFlow Data:** A set of Python functionalities provided by TensorFlow under the ``tf.data`` module, especially with ``Dataset`` objects. Features include:
 - Prefetching transformations to overlap preprocessing and model execution with the ``tf.data.Dataset.prefetch`` transformation
 - Cache data in memory during the first epoch using the ``tf.data.Dataset.cache``
 - Parallelize data reading with ``tf.data.Dataset.interleave``
 - Can handle various data formats, such as TFRecords, CSVs, or image data



TensorFlow Transform

- **TensorFlow Transform:** This library is part of TensorFlow Extended (TFX), and allows you to define preprocessing steps at the training time that can be applied consistently at inference time
 - You can perform transformations prior to training and during, and avoid training-serving skew – ensuring the same transformations are applied at both training and inference
 - Transformation steps include: analyzing and transforming training data, transforming validation data, producing metadata, and serving data
 - Can integrate with Apache Beam to process data at scale



Vertex AI Feature Store

- Vertex AI Feature Store is a fully managed, centralized repository within Google Cloud designed to store, share, and manage features across various models and features
 - When you're training a model, users can define features and then import feature values from other sources
 - You can retrieve features in real time to perform fast online predictions
 - Reduces duplication of effort in feature engineering, offering a single source of truth for all ML features
 - Integrates seamlessly with other Vertex AI services



Question #8

- You are the data scientist for your company. You have a dataset which has all categorical features. You trained a model using some algorithms. With some algorithms this data is giving good results but when you change the algorithm the performance is getting reduced. Which data transformation strategy would likely improve the performance of your model?
 - A. Write your data in TFRecords.
 - B. Create a feature cross with categorical features.
 - C. Use one-hot encoding on all categorical features.
 - D. Oversample the features.



Question #8

- You are the data scientist for your company. You have a dataset which has all categorical features. You trained a model using some algorithms. With some algorithms this data is giving good results but when you change the algorithm the performance is getting reduced. Which data transformation strategy would likely improve the performance of your model?
 - A. Write your data in TFRecords.
 - B. Create a feature cross with categorical features.
 - C. Use one-hot encoding on all categorical features.
 - D. Oversample the features.

Not all algorithms can handle categorical features; many work better with numerical data. One-hot encoding converts the features to numeric.



Question #9

- You are transforming a complete dataset before model training. Your model accuracy is 99 percent in training, but in production its accuracy is 66 percent. What is a possible way to improve the model in production?
 - A. Apply transformation during model training.
 - B. Perform data normalization.
 - C. Remove missing values.
 - D. Use `tf.Transform` for creating production pipelines for both training and serving.



Question #9

- You are transforming a complete dataset before model training. Your model accuracy is 99 percent in training, but in production its accuracy is 66 percent. What is a possible way to improve the model in production?
 - A. Apply transformation during model training.
 - B. Perform data normalization.
 - C. Remove missing values.
 - D. Use `tf.Transform` for creating production pipelines for both training and serving.

We appear to have training-serving skew. Using a `tf.Transform` pipeline helps ensure the same steps are applied during model training as serving. This helps avoid discrepancies resulting from differences in how data is processed between training and production stages.



Model building, training, tuning, evaluation





Modeling techniques

The simplest type of neural network is an artificial neural network (ANN) that has one hidden layer, common for supervised learning problems with structured data. For more complex problems, it is common to use deep neural networks (DNNs) – ANNs with increased numbers of hidden layers.

- **Convolutional Neural Network (CNN):** Designed for image input and image classification
- **Recurrent Neural Network (RNN):** Designed to operate on sequences of data
 - Also very common for NLP problems, because text is sequential
 - Most common type is a long short-term memory (LSTM) network



Cost functions

The cost function quantifies how much error the current model has.

- Regression problems:
 - Use mean-squared error (MSE) or mean absolute error (MAE)
- Binary classification:
 - Binary cross-entropy, categorical hinge loss, or squared hinge loss (Keras)
- Multiclass classification:
 - Categorical cross-entropy if one sample can have multiple classes or labels
 - Sparse categorical cross-entropy if the classes are mutually exclusive; one sample can belong to exactly one class



Question #10

- Your team needs to build a model that predicts whether images contain a driver's license, passport, or credit card. The data engineering team already built the pipeline and generated a dataset composed of 20,000 images with driver's licenses, 2,000 images with passports, and 2,000 images with credit cards. You now have to train a model with the following label map: ['drivers_license', 'passport', 'credit_card']. Which loss function should you use?
 - A. Categorical hinge
 - B. Binary cross-entropy
 - C. Categorical cross-entropy
 - D. Sparse categorical cross-entropy



Question #10

- Your team needs to build a model that predicts whether images contain a driver's license, passport, or credit card. The data engineering team already built the pipeline and generated a dataset composed of 20,000 images with driver's licenses, 2,000 images with passports, and 2,000 images with credit cards. You now have to train a model with the following label map: ['drivers_license', 'passport', 'credit_card']. Which loss function should you use?
 - A. Categorical hinge
 - B. Binary cross-entropy
 - C. Categorical cross-entropy
 - D. Sparse categorical cross-entropy

This is a multiclass classification problem where the labels are mutually exclusive.



Question #11

- You work for a bank and are building a random forest model for fraud detection. You have a dataset that includes transactions, of which 1% are identified as fraudulent. Which data transformation strategy would likely improve the performance of your classifier?
 - A. Write your data in TFRecords.
 - B. Z-normalize all the numeric features.
 - C. Oversample the fraudulent transaction 10 times.
 - D. Use one-hot encoding on all categorical features.



Question #11

- You work for a bank and are building a random forest model for fraud detection. You have a dataset that includes transactions, of which 1% are identified as fraudulent. Which data transformation strategy would likely improve the performance of your classifier?
 - A. Write your data in TFRecords.
 - B. Z-normalize all the numeric features.
 - C. Oversample the fraudulent transaction 10 times.
 - D. Use one-hot encoding on all categorical features.

The issue here is class imbalance (1% fraud). Good strategies to deal with this include oversampling and undersampling.

Overfitting and underfitting

In building an ML model, there is a tradeoff between bias and variance.

- Underfitting is where prediction error is not minimized
 - This type of model typically has high bias but low variance
- Overfitting is where the model fits the training dataset very well but poorly predicts on new data
 - This type of model typically has low bias but high variance
- Ideally we want to make the model fit well on the data but not lose generality
- Two common solutions to this problem are **early stopping** and **regularization**



Early stopping

- Early stopping is a method that involves ending training before training loss finishes decreasing
- Stop training the model when the loss on a validation dataset starts to increase; i.e., generalization diminishes
- Decreases chance of overfitting to training data



Regularization

Regularization refers to any method that reduces overfitting. It is done by assigning a penalty on a model's complexity.

- Early stopping is a method that involves ending training before training loss finishes decreasing
- Stop training the model when the loss on a validation dataset starts to increase; i.e., generalization diminishes
- Other popular types of regularization include: L1 regularization, and L2 regularization

L1 vs L2 Regularization

- **L1 regularization:**

- Adds a penalty equal to the absolute value of the magnitude of coefficients to the loss function
- Regularization term is the sum of the absolute values of all the feature weights, multiplied by the regularization strength λ
- It can zero out some of the model weights, performing feature selection
- Robust to outliers

- **L2 regularization:**

- Adds a penalty equal to the square of the magnitude of coefficients to the loss function
- Regularization term is the sum of the squares of all the feature weights multiplied by the regularization strength λ
- It tends to shrink the coefficients relatively evenly so can help with multicollinearity
- Improves generalization in linear models



Data leakage

Data leakage is a situation that occurs when the model has been exposed to test data during training. Generally, we will see great performance on the testing data but the model will not perform well when exposed to new, unseen data.

- This can happen if we make the target variable a feature by mistake
- If we pre-process data BEFORE splitting into training, testing, and validation sets, the model can learn not only the training data but the test data
- It is common in time-series if we split the data randomly rather than sequentially; effectively the model can use data from the future to learn about the present



Hyperparameter tuning

Hyperparameter tuning attempts to find the best version of a model by running many training jobs, utilizing the algorithm and a specified range of hyperparameters.

- Hyperparameter tuning attempts to find the best version of a model by running many training jobs, utilizing the algorithm and a specified range of hyperparameters.
- Hyperparameters are the external configurations of the model that are not learned from the data but set prior to the training process, but require manual specification
- An example is the number of layers in a neural network model and activation functions
- We will show some examples of algorithm hyperparameters



Vertex AI Vizier

Vertex AI Vizier is an automated hyperparameter tuning and optimization service included in Vertex AI.

- It automates the process, allowing users to define a search space for hyperparameters (incl. continuous and discrete hyperparameters) using algorithms such as Bayesian optimization (default), random search, and grid search
- Can parallelize hyperparameter evaluations, running multiple trials concurrently
- Integrates with Vertex AI Training, Prediction, Pipelines, etc.
- Provides cost-aware optimization strategies, e.g. intelligently scheduling trials, dynamically adjusting parallelism, early dropping of unproductive trials



Batch Size

- The number of training samples used in one iteration of model training.
- Essentially, the number of samples the model is exposed to before the model's weights are updated.
 - A smaller batch size can lead to a faster convergence as the model is updated more frequently, but higher variance in training performance
 - Larger batch size provides more stable estimates but it requires more memory and can lead to slower convergence



Epochs

- The number of times that we run through the entire training dataset. When a model has seen every sample in the dataset once, it has completed one epoch.
 - Fewer epochs can prevent overfitting, but might result in underfitting because the model hasn't learned enough from the training data to make accurate predictions.
 - More epochs can lead to overfitting



Learning Rate

- The size of the steps that the model takes during optimization.
- It controls how much the model's weights are adjusted with respect to the loss gradient.
 - A lower learning rate will lead to more stable convergence, but it can make the training process slower and potentially get stuck in local minima
 - A higher learning rate allows the model to learn faster, but it can lead to unstable training or divergence



Question #12

- You are a data scientist training a deep neural network. During batch training of the neural network, you notice that there is an oscillation in the loss. How should you adjust your model to ensure that it converges?
 - A. Increase the size of the training batch.
 - B. Decrease the size of the training batch.
 - C. Increase the learning rate hyperparameter.
 - D. Decrease the learning rate hyperparameter.

Question #12

- You are a data scientist training a deep neural network. During batch training of the neural network, you notice that there is an oscillation in the loss. How should you adjust your model to ensure that it converges?
 - A. Increase the size of the training batch.
 - B. Decrease the size of the training batch.
 - C. Increase the learning rate hyperparameter.
 - D. Decrease the learning rate hyperparameter.

Oscillating loss implies that the model is overcorrecting. This is probably due to the learning rate being too high. Decreasing it will stabilize the training process. Increasing batch size could diminish accuracy, and decreasing batch size can increase loss oscillation.



Question #13

- You started working on a classification problem with time series data and achieved an area under the receiver operating characteristic curve (AUC ROC) value of 99% for training data after just a few experiments. You haven't explored using any sophisticated algorithms or spent any time on hyperparameter tuning. What should your next step be to identify and fix the problem?
 - A. Address the model overfitting by using a less complex algorithm
 - B. Address data leakage by applying nested cross-validation during model training
 - C. Address data leakage by removing features highly correlated with the target value
 - D. Address the model overfitting by tuning the hyperparameters to reduce the AUC ROC value



Question #13

- You started working on a classification problem with time series data and achieved an area under the receiver operating characteristic curve (AUC ROC) value of 99% for training data after just a few experiments. You haven't explored using any sophisticated algorithms or spent any time on hyperparameter tuning. What should your next step be to identify and fix the problem?
 - A. Address the model overfitting by using a less complex algorithm
 - B. Address data leakage by applying nested cross-validation during model training
 - C. Address data leakage by removing features highly correlated with the target value
 - D. Address the model overfitting by tuning the hyperparameters to reduce the AUC ROC value

This is a sign that the model is overfitting and 99% implies data leakage; cross-validation during model training will help to prevent data leakage and overfitting to the training data.



Question #14

- You need to write a generic test to verify whether DNN models automatically released by your team have a sufficient number of parameters to learn the task for which they were built. What should you do?
 - A. Train the model for a few iterations, and check for NaN values
 - B. Train the model for a few iterations, and verify that the loss is constant
 - C. Train a simple linear model, and determine if the DNN model outperforms it
 - D. Train the model with no regularization, and verify that the loss function is close to zero



Question #14

- You need to write a generic test to verify whether DNN models automatically released by your team have a sufficient number of parameters to learn the task for which they were built. What should you do?
 - A. Train the model for a few iterations, and check for NaN values
 - B. Train the model for a few iterations, and verify that the loss is constant
 - C. Train a simple linear model, and determine if the DNN model outperforms it
 - D. Train the model with no regularization, and verify that the loss function is close to zero

We expect the loss function to decrease if it has enough parameters to learn and memorize the task. Outperforming a linear model does not guarantee a model has enough parameters to learn tasks with non-linear data representations.

Question #15

- Your training, validation, and test sets have similar distributions. You have sub-selected the most relevant features and trained a neural network in TensorFlow. TensorBoard plots show the training loss oscillating around 0.9, with the validation loss higher than the training loss by 0.3. You want to update the training regime to maximize the convergence of both losses and reduce overfitting. What should you do?
 - A. Decrease the learning rate to fix the validation loss, and increase the number of training epochs to improve the convergence of both losses.
 - B. Decrease the learning rate to fix the validation loss, and increase the number and dimension of the layers in the network to improve the convergence of both losses.
 - C. Introduce L1 regularization to fix the validation loss, and increase the learning rate and the number of training epochs to improve the convergence of both losses.
 - D. Introduce L2 regularization to fix the validation loss.



Question #15

- Your training, validation, and test sets have similar distributions. You have sub-selected the most relevant features and trained a neural network in TensorFlow. TensorBoard plots show the training loss oscillating around 0.9, with the validation loss higher than the training loss by 0.3. You want to update the training regime to maximize the convergence of both losses and reduce overfitting. What should you do?
 - A. Decrease the learning rate to fix the validation loss, and increase the number of training epochs to improve the convergence of both losses.
 - B. Decrease the learning rate to fix the validation loss, and increase the number and dimension of the layers in the network to improve the convergence of both losses.
 - C. Introduce L1 regularization to fix the validation loss, and increase the learning rate and the number of training epochs to improve the convergence of both losses.
 - D. Introduce L2 regularization to fix the validation loss.

Changing the learning rate does not reduce overfitting. Additionally, increasing the number of training epochs is not expected to improve losses significantly. L2 regularization prevents overfitting.



Q & A





What is TensorFlow?

TensorFlow is an open-source machine learning framework developed by Google for building and deploying machine learning platforms.

- It allows users to build and customize machine learning models for diverse tasks and applications
- It supports distributed computing, enabling the training and deployment of large-scale ML models across CPUs or GPUs
- It is optimized for performance, with support for hardware acceleration using GPUs and TPUs
- Includes TensorBoard, a visualization tool for monitoring ML models and visualizing training metrics
- Widely used for deep learning tasks and NLP



CPUs & GPUs

We need to understand the different hardware components in Google Cloud for computing needs.

- **Central Processing Units (CPUs):** These are versatile and suitable for general-purpose computing tasks, rapid prototypes, or models that train fast. They are usually more cost-effective than hardware like GPUs and TPUs, but are not necessarily optimal for latency-sensitive tasks or for parallel processing.
- **Graphics Processing Units (GPUs):** GPUs are designed for highly parallel processing tasks (e.g. ML, deep learning, simulations). They can significantly accelerate the training process, but can be more expensive and more complex and difficult to manage than CPUs, because developers need to write code specifically optimized for GPU architecture
 - Great when we have many custom TensorFlow operations



GPUs Continued

If you have written your training code to use GPUs, then you may configure your worker pool to use one or more GPUs on each VM.

- You must use an A2, N1, or G2 machine type
- The following types are supported:
- Reference: <https://cloud.google.com/vertex-ai/docs/training/configure-compute#gpu-compatibility-table>

- NVIDIA_H100_80GB
- NVIDIA_A100_80GB
- NVIDIA_TESLA_A100 (NVIDIA A100 40GB)
- NVIDIA_TESLA_K80
- NVIDIA_TESLA_P4
- NVIDIA_TESLA_P100
- NVIDIA_TESLA_T4
- NVIDIA_TESLA_V100
- NVIDIA_L4



GPU Constraints

- In your `WorkerPoolSpec``, specify the type of GPU that you want to use in the ``machineSpec.acceleratorType`` field and number of GPUs that you want each VM in the worker pool to use in the ``machineSpec.acceleratorCount`` field
- The type of GPU you choose must be available in the location where you are performing custom training
- You can only use certain numbers of GPUs in your configuration (e.g. 2 or 4 NVIDIA_TESLA_T4 GPUs on a VM, but not 3)
- You must make sure your GPU configuration provides sufficient virtual CPUs and memory to the machine type used (e.g. if you use the n1-standard-32 machine type in your worker pool, each VM has 32 virtual CPUs and 120 GB of memory)
 - Since each NVIDIA_TESLA_V100 GPU can provide up to 12 virtual CPUs and 76 GB of memory, you must use at least 4 GPUs for each n1-standard-32 VM to support its requirements
- Reference: <https://cloud.google.com/vertex-ai/docs/training/configure-compute#gpu-compatibility-table>



GPU Compatibility

Valid numbers of GPUs for each machine type							
Machine type	NVIDIA_A100_80GB	NVIDIA_TESLA_A100	NVIDIA_TESLA_K80	NVIDIA_TESLA_P4	NVIDIA_TESLA_P100	NVIDIA_TESLA_T4	NVIDIA_TESLA_V100
a2-ultragpu-1g	1						
a2-ultragpu-2g	2						
a2-ultragpu-4g	4						
a2-ultragpu-8g	8						
a2-highgpu-1g		1					
a2-highgpu-2g		2					
a2-highgpu-4g		4					
a2-highgpu-8g		8					
a2-megagpu-16g		16					
n1-standard-4			1, 2, 4, 8	1, 2, 4	1, 2, 4	1, 2, 4	1, 2, 4, 8
n1-standard-8			1, 2, 4, 8	1, 2, 4	1, 2, 4	1, 2, 4	1, 2, 4, 8
n1-standard-16			2, 4, 8	1, 2, 4	1, 2, 4	1, 2, 4	2, 4, 8
n1-standard-32			4, 8	2, 4	2, 4	2, 4	4, 8
n1-standard-64				4		4	8

Valid numbers of GPUs for each machine type							
Machine type	NVIDIA_A100_80GB	NVIDIA_TESLA_A100	NVIDIA_TESLA_K80	NVIDIA_TESLA_P4	NVIDIA_TESLA_P100	NVIDIA_TESLA_T4	NVIDIA_TESLA_V100
n1-highcpu-16			2, 4, 8	1, 2, 4	1, 2, 4	1, 2, 4	2, 4, 8
n1-highcpu-32			4, 8	2, 4	2, 4	2, 4	4, 8
n1-highcpu-64			8	4	4	4	8
n1-highcpu-96				4		4	8
g2-standard-4							1
g2-standard-8							1
g2-standard-12							1
g2-standard-16							1
g2-standard-24							2
g2-standard-32							1
g2-standard-48							4
g2-standard-96							8



TPUs, Edge Devices

- **Tensor Processing Units (TPUs):** Custom-built by Google for machine learning workloads. They are specialized and primarily used for machine learning workloads and have outstanding performance in deep learning tasks, but may not be suitable for general-purpose computing tasks.
- **Edge devices:** These are used for performing computing tasks at or near the data source, reducing latency and bandwidth requirements. They can be customized to suit specific uses, but typically have limited computing resources compared to cloud-based solutions.



TPU Configuration, Use Cases

- The following TPU configurations are available:
 - A single TPU
 - A TPU pod (a group of TPU devices)
 - A TPU slice (a subdivision of a TPU pod)
 - A TPU VM
- TPUs can be dramatically faster computationally than GPUs (and especially CPUs) but are not suited for cases with custom TensorFlow operations or with frequent if/else conditional branching, or sparse data with few nonzero values



TensorFlow Enterprise, TensorFlow Extended

These are both offerings from Google related to machine learning, though they serve different purposes and target different aspects of ML development and deployment.

- **TensorFlow Enterprise** is a service offering from Google Cloud, designed to provide enterprise-level support for TensorFlow
 - It provides access to official Google Cloud support, service-level agreements, and support for deploying and managing TensorFlow in production environments
- **TensorFlow Extended (TFX)** is an end-to-end machine learning platform primarily used for building, managing, and deploying machine learning pipelines
 - Includes open-source components for various tasks (e.g. data ingestion, feature engineering, model training) and promotes best MLOps practices



Distributed training strategies

Can do data parallelism or model parallelism. Can use the `tf.distribute.Strategy` TensorFlow API to distribute training across multiple CPUs, GPUs, or TPUs.

- **MirroredStrategy:** Designed for synchronous training on multiple GPUs on the same machine
- **MultiWorkerMirroredStrategy:** Extends `MirroredStrategy` for synchronous training across multiple workers (machines), each possibly having multiple GPUs
- **TPUStrategy:** Designed for synchronous training on multiple TPUs
- **ParameterServerStrategy:** Designed for asynchronous training across multiple workers; some machines designated as workers and some as parameter servers
 - Particularly suited for large scale models and data parallelism



Question #16

- You are working for a start-up on several projects with TensorFlow. You need to increase the performance of the training and you are already using caching and prefetching. You want to use GPU for training but you have to use only one machine to be cost-effective. Which of the following tf distribution strategies should you use?
 - A. MirroredStrategy
 - B. MultiWorkerMirroredStrategy
 - C. TPUStrategy
 - D. ParameterServerStrategy



Question #16

- You are working for a start-up on several projects with TensorFlow. You need to increase the performance of the training and you are already using caching and prefetching. You want to use GPU for training but you have to use only one machine to be cost-effective. Which of the following tf distribution strategies should you use?
 - A. **MirroredStrategy**
 - B. MultiWorkerMirroredStrategy
 - C. TPUStrategy
 - D. ParameterServerStrategy

The keys are that we want to use a GPU, and that we have to use only one machine.



Question #17

- Your team is building a convolutional neural network for an image segmentation problem on-prem on a CPU-only machine. It takes a long time to train, so you want to speed up the process by moving to the cloud. You experiment with VMs on Google Cloud to use better hardware. You do not have any code for manual placements and have not used any custom transformations. What hardware should you use?
 - A. A deep learning VM with n1-standard-2 machine with 1 GPU
 - B. A deep learning VM with more powerful e2-highCPU-16 machines
 - C. A VM with 8 GPUs
 - D. A VM with 1 TPU



Question #17

- Your team is building a convolutional neural network for an image segmentation problem on-prem on a CPU-only machine. It takes a long time to train, so you want to speed up the process by moving to the cloud. You experiment with VMs on Google Cloud to use better hardware. You do not have any code for manual placements and have not used any custom transformations. What hardware should you use?
 - A. A deep learning VM with n1-standard-2 machine with 1 GPU
 - B. A deep learning VM with more powerful e2-highCPU-16 machines
 - C. A VM with 8 GPUs
 - D. A VM with 1 TPU

n1-standard-2 is too small for GPUs. 1 TPU instead of 8 GPUs is usually a better choice since we don't have manual placements. Option B is still using CPUs.



Question #18

- You are training a deep learning model for object detection. It is taking too long to converge, so you are trying to speed up the training. While you are trying to launch an instance (with GPU) with Deep Learning VM Image, you get an error that the “NVIDIA_TESLA_V100 was not found.” What could be the problem?
 - A. GPU was not available in the selected region.
 - B. GPU quota was not sufficient.
 - C. Preemptible GPU quota was not sufficient.
 - D. GPU did not have enough memory.



Question #18

- You are training a deep learning model for object detection. It is taking too long to converge, so you are trying to speed up the training. While you are trying to launch an instance (with GPU) with Deep Learning VM Image, you get an error that the “NVIDIA_TESLA_V100 was not found.” What could be the problem?
 - A. GPU was not available in the selected region.
 - B. GPU quota was not sufficient.
 - C. Preemptible GPU quota was not sufficient.
 - D. GPU did not have enough memory.

An error of “not found” implies that the GPU is not available in the region.



Question #19

- You are building a product that will accurately throw a ball into a basketball net. This should work no matter what it is placed on the court. You have created a very large TensorFlow model (size more than 90 GB) based on thousands of hours of video. The model uses custom operations, and it has optimized the training loop to not have any I/O operations. What are your hardware options to train this model?
 - A. Use a TPU slice because the model is very large and has been optimized to not have any I/O operations
 - B. Use a TPU pod because the model size is > 50 GB
 - C. Use a GPU-only instance
 - D. Use a CPU-only instance to build your model



Question #19

- You are building a product that will accurately throw a ball into a basketball net. This should work no matter what it is placed on the court. You have created a very large TensorFlow model (size more than 90 GB) based on thousands of hours of video. The model uses custom operations, and it has optimized the training loop to not have any I/O operations. What are your hardware options to train this model?
 - A. Use a TPU slice because the model is very large and has been optimized to not have any I/O operations
 - B. Use a TPU pod because the model size is > 50 GB
 - C. Use a GPU-only instance
 - D. Use a CPU-only instance to build your model

TPU cannot be used, because of the custom TensorFlow operations. CPUs will be very slow, so GPU is the way to go here.



Question #20

- You want to use GPUs for training your models that need about 50 GB of memory. What hardware options do you have?
 - A. n1-standard-64 with 8 NVIDIA_TESLA_P100
 - B. e2-standard-32 with 4 NVIDIA_TESLA_P100
 - C. n1-standard-32 with 3 NVIDIA_TESLA_P100
 - D. n2d-standard-32 with 4 NVIDIA_TESLA_P100



Question #20

- You want to use GPUs for training your models that need about 50 GB of memory. What hardware options do you have?
 - A. n1-standard-64 with 8 NVIDIA_TESLA_P100
 - B. e2-standard-32 with 4 NVIDIA_TESLA_P100
 - C. n1-standard-32 with 3 NVIDIA_TESLA_P100
 - D. n2d-standard-32 with 4 NVIDIA_TESLA_P100

Use some of the known GPU constraints. GPUs are only supported by a2 and n1 instances. Additionally, you cannot have 3 GPUs in an instance.



Batch vs online serving

An ML model can provide predictions in two ways: offline (batch) prediction and online prediction

- **Batch (Offline) Prediction:** This is typical when you have data collected over a period but then processed in a single batch
 - Use cases: historical analysis, demand forecasting, cases when computational resources are limited
- **Online Prediction:** Happens in near real-time as the data is received; model is always running and predictions made on-demand
 - Use cases: real-time bidding/recommendations, dynamic pricing, fraud detection



Model monitoring

- Conceptually a model may perform well on the validation and test sets, but over time after deployment if the underlying data/environment change, the model will become less useful over time
- An example is **concept drift** – this is when the underlying assumptions of the model change
 - Example: Spammers modify their emails to bypass existing spam filters
- We can monitor models for skew and drift:
 - **Training-serving skew** is when there is a difference in the input feature distribution in production, compared to training data
 - **Prediction drift** is when the input's statistical distribution changes in production over time



Model monitoring tools

The following are some tools that can be used for tracking metrics during training:

- **TensorFlow Profiler:** Can be used for analyzing and optimizing TensorFlow models' performance
 - Can be used to profile the performance of Vertex AI training jobs and visualize results in Vertex TensorBoard
- **What-If Tool:** A visualization tool for exploring and understanding ML models' behavior, predictions, and biases using interactive scenarios
 - Displays interactive visualization within the notebook



Vertex Explainable AI

- Part of model monitoring, essential in some business applications, is being able to explain model predictions
- Vertex Explainable AI is a feature of Vertex AI that provides tools for explaining and interpreting your model's predictions and behaviors
- Supported services include AutoML image and tabular models or custom-trained TensorFlow image and tabular models
- The focus of Vertex Explainable AI is around instance-level feature attribution



Feature attribution

- Vertex Explainable AI offers three methods for feature attributions:
 - Sampled Shapley
 - Estimates feature importance by assigning credit for the outcome to each feature
 - Integrated gradients
 - Mostly used for deep neural networks with image use cases; integrates gradients along the path from a baseline input to the actual input
 - XRAI (Explanation with Ranked Area Integrals)
 - Combines the integrated gradients method with additional steps to determine which regions of the image contributed the most to the outcome



Question #21

- When there is a difference in the input feature distribution between the training data and the data in production, what is this called?
 - A. Distribution drift
 - B. Feature drift
 - C. Training-serving skew
 - D. Concept drift



Question #21

- When there is a difference in the input feature distribution between the training data and the data in production, what is this called?
 - A. Distribution drift
 - B. Feature drift
 - C. Training-serving skew
 - D. Concept drift

This question tests knowledge of the definitions of these terms.



Question #22

- You are a data scientist at a startup and your team is working on a number of ML Projects. Your team trained a TensorFlow deep neural network model for image recognition that works well and is about to be rolled out in production. You have been asked to demonstrate the inner workings of the model. What explainability technique would you use on Google Cloud?
 - A. Sampled Shapley
 - B. Integrated gradient
 - C. PCA
 - D. What-If Tool analysis



Question #22

- You are a data scientist at a startup and your team is working on a number of ML Projects. Your team trained a TensorFlow deep neural network model for image recognition that works well and is about to be rolled out in production. You have been asked to demonstrate the inner workings of the model. What explainability technique would you use on Google Cloud?
 - A. Sampled Shapley
 - **B. Integrated gradient**
 - C. PCA
 - D. What-If Tool analysis

The key here is our model is for image recognition. Integrated gradient is the best tool for deep neural networks on images.



Pipelines & Security





Identity and Access Management

Identity and Access Management (IAM) is a framework and set of tools used in Google Cloud used for defining and managing permissions for users, groups, and services.

- Can be managed at the project level or the resource level
- The following IAM roles can be used in Vertex AI:
 - Predefined (e.g. Vertex AI Administrator, Vertex AI User)
 - Basic (Owner, Editor, Viewer)
 - These are common roles to all GCP services
 - Custom
 - Can set own sets of permissions – usually the minimum necessary are provided



User-managed vs. managers notebooks

The two types of Vertex AI notebooks are as follows:

- **User-Managed:** Customizable, ideal for users who require lots of control over their environment
 - Choose one framework for your JupyterLab environment (i.e. R, TensorFlow, PyTorch)
 - Best option if specific networking and security needs required
- **Managed:** Google Cloud-managed notebooks
 - These integrate with BigQuery and Cloud Storage in the Jupyter Lab environment
 - Can be scheduled for an execution
 - Includes automatic shutdown for idle instances
 - Can process data quickly by running a notebook on a Dataproc cluster or serverless Spark cluster
 - Can be connected to a VPC network in the same project or a shared network that you have access to
 - VPC Service Controls can configure access to specific services



Google Cloud Data Loss Prevention (DLP)

The DLP API is a powerful, fully managed service designed to help users discover, classify, and protect sensitive information. It can de-identify sensitive data in text content; it can detect sensitive data and use de-identification transformations to mask, delete, or obscure the data. Designed to scale automatically with demand.

- Includes risk analysis methods before de-identification to help assess and prioritize risks related to data storage and processing practices
- De-identification techniques include:
 - Masking (e.g. replacing characters with symbols such as asterisks (*))
 - Tokenization (e.g. cryptographic hashing to replace instances with tokens)
 - Encryption (replacing data with randomly generated or predetermined keys)
 - Bucketing (replacing more identifiable values with less identifiable ones)

Federated learning

Federated learning is a machine learning approach that enables a model to be trained across multiple decentralized devices holding data samples without needing to exchange them

- Instead of aggregating data in a central server, federated learning brings the model directly to the data source (e.g. mobile devices, hospital servers)
- This inherently protects users' privacy; additional techniques like differential privacy and secure multi-party computation can further secure this process
 - Also reduces the need for constant internet connectivity and data transmission, reducing latency and power consumption
- Devices train the model on their local data, generating model updates; the central server collects the local updates, and then the updated global model is sent back to all devices



Orchestration frameworks

- There are multiple steps in a complex ML workflow, including but not limited to: data collection, pre-processing, model training, evaluation, deployment, and monitoring
- You need an orchestrator to help manage, automate, and optimize these processes
- The orchestrator runs the pipeline sequentially, moving from one step to another based on pre-specified criteria
- There are multiple options, including Kubeflow Pipelines, Vertex AI Pipelines, and Cloud Composer



Kubeflow



Kubeflow is an open-source platform designed to make it easier to deploy, manage, and scale ML workflows on Kubernetes.

- Kubernetes can leverage Kubernetes' container orchestration capabilities to scale ML workloads based on demand
- Provides a consistent environment for ML development and deployment across different environments, including on-prem, cloud, or hybrid cloud environments
- Enables the creation of end-to-end ML pipelines
- Can serve predictions via REST APIs



Vertex AI Pipelines

Vertex AI Pipelines is a managed service on GCP, as part of Vertex AI, that enables users to create, deploy, and manage end-to-end ML workflows.

- Provides tools for orchestrating complex ML workflows (i.e. data ingestion, preprocessing, model training, evaluation, and deployment)
- Supports versioning of pipeline components, offers visualization tools and monitoring dashboards
- Users can create custom components using Python code
- Integrates seamlessly with other Vertex AI and GCP services (e.g. Vertex AI Datasets, BigQuery, Cloud Storage, Dataflow, Pub/Sub)



Cloud Composer

Cloud Composer is a fully managed workflow orchestration service on GCP based on Apache Airflow.

- It enables the automation of complex workflows and data pipelines, orchestrating tasks and dependencies across different services and systems
- Like Vertex AI Pipelines, integrates seamlessly with other GCP services
- Workflows in Cloud Composer are represented as Das (Directed Acyclic Graphs), where nodes represent tasks and edges represent dependencies between tasks
- Provides Apache Airflow deployments for managing workflows
- Commonly used for workflow automation, ETL pipelines, or data processing



Scheduling workflows with Kubeflow

Kubeflow Pipelines can be invoked using the following services:

- Cloud Scheduler
- Pub/Sub and Cloud Functions to respond to an event (e.g. new files in a Cloud Storage bucket)
- Can use Cloud Composer or Cloud Data Fusion if part of a bigger data and process workflow
- Cloud Build



When to use each pipeline

- TFX is recommended if you are already using TensorFlow and using lots of structured/textual data
 - Useful for building end-to-end ML workflows within TensorFlow ecosystems
- Cloud Composer often used to orchestrate data-driven workflows (e.g. ETL, batch processing) but is always active
- Vertex AI Pipelines is designed for end-to-end ML development and deployment, with support for monitoring and lineage
- Can use Kubeflow Pipelines with Vertex AI Pipelines for deploying, orchestrating, and managing the workflow
 - Well-suited for hybrid and multi-cloud environments
 - Allows users to define custom containerized components and execution on Kubernetes clusters



Triggers and pipeline schedules

There are various ways that we can trigger a training or prediction job on Vertex AI:

- **Cloud Scheduler** can be used to set up cron jobs to schedule these jobs
- The Jupyter Notebook with Vertex AI managed notebooks can be used to schedule these jobs
- **Cloud Functions** are event-driven, serverless, stateless functions
- **Cloud Build** is a CI/CD offering that you can be used for building Dockerfiles, retraining models, pushing for custom training, etc.
- **Cloud Run** can be used for running containerized applications



Lineage



Lineage refers to the tracking and recording of the history and dependencies of artifacts and processes involved in ML workflows. Lots of support in Vertex AI Pipelines.

- **Data lineage** tracks the origin and processing history of datasets
- **Model lineage** tracks the training data, hyperparameters, training configurations, and model versions generated
- **Code lineage** tracks the execution path and dependencies of code components (e.g. scripts, modules, functions)
- **Artifact lineage** tracks all the factors that resulted in an artifact, including data, models, code, and configurations



Vertex ML Metadata

Vertex ML Metadata is a service for management metadata in ML workflows; helps track and organize metadata

- Tracks and stores metadata related to ML artifacts like datasets, models, and training runs. Examples are versioning, lineage, and dependencies
- Enables tracking and management of ML experiments. Examples are hyperparameters, metrics, and outcomes
- Maintains version history for ML artifacts
- Facilitates model monitoring by capturing metadata around predictions, drift detection, data quality, etc.



Question #23

- A national hospital is leveraging GCP and a cell phone app to build an ML model to forecast heart attacks based on age, gender, exercise, heart rate, blood pressure, and more. Since the health data is highly sensitive personal information and cannot be stored in cloud databases, how should you train and deploy the ML model?
 - A. IoT with data encryption
 - B. Federated learning
 - C. Encrypted BQML
 - D. DLP API



Question #23

- A national hospital is leveraging GCP and a cell phone app to build an ML model to forecast heart attacks based on age, gender, exercise, heart rate, blood pressure, and more. Since the health data is highly sensitive personal information and cannot be stored in cloud databases, how should you train and deploy the ML model?
 - A. IoT with data encryption
 - **B. Federated learning**
 - C. Encrypted BQML
 - D. DLP API

All the data would be collected, and the model would be trained with algorithms across multiple decentralized edge devices without exchanging them.



Question #24

- You developed a tree-based model based on an extensive feature set of user behavioral data. The tree model has been in production for 6 months. New regulations were just introduced that require anonymizing PII, which you have in your feature set using the DLP API. You want to update your model pipeline to adhere to the new regulations while minimizing a reduction in model performance. What should you do?
 - A. Redact the features containing PII data, and train the model from scratch.
 - B. Mask the features containing PII data, and tune the model from the last checkpoint.
 - C. Use key-based hashes to tokenize the features containing PII data, and train the model from scratch.
 - D. Use deterministic encryption to tokenize the features containing PII data, and tune the model from the last checkpoint.



Question #24

- You developed a tree-based model based on an extensive feature set of user behavioral data. The tree model has been in production for 6 months. New regulations were just introduced that require anonymizing PII, which you have in your feature set using the DLP API. You want to update your model pipeline to adhere to the new regulations while minimizing a reduction in model performance. What should you do?
 - A. Redact the features containing PII data, and train the model from scratch.
 - B. Mask the features containing PII data, and tune the model from the last checkpoint.
 - C. Use key-based hashes to tokenize the features containing PII data, and train the model from scratch.
 - D. Use deterministic encryption to tokenize the features containing PII data, and tune the model from the last checkpoint.

Masking does not enforce referential integrity. Additionally, deterministic encryption is reversible; anonymization requires irreversibility. Redaction is likely to cause a drop in performance. Finally, tuning the existing model is not appropriate because the model may have memorized sensitive information.



Question #25

- You created an ML model and want to use the data to refresh your model as soon as new data is available in a Google Cloud Storage bucket. As part of your CI/CD workflow, you want to automatically run a Kubeflow Pipelines training job on GKE. How should you design this workflow with the least effort and in the most managed way?
 - A. Configure a Cloud Storage trigger to send a message to a Pub/Sub topic when a new file is available in a storage bucket. Use a Pub/Sub-triggered Cloud Function to start the training job on GKE.
 - B. Use Cloud Scheduler to schedule jobs at a regular interval. For the first step of the job, check the time stamp of objects in your Cloud Storage bucket to see if there are no new files since the last run.
 - C. Use App Engine to create a lightweight Python client that continuously polls Cloud Storage for new files. As soon as a file arrives, initiate the Kubeflow Pipelines job on GKE.
 - D. Configure your pipeline with Dataflow, which saves the files in Cloud Storage. After the file is saved, you can start the job on GKE.



Question #25

- You created an ML model and want to use the data to refresh your model as soon as new data is available in a Google Cloud Storage bucket. As part of your CI/CD workflow, you want to automatically run a Kubeflow Pipelines training job on GKE. How should you design this workflow with the least effort and in the most managed way?
 - A. Configure a Cloud Storage trigger to send a message to a Pub/Sub topic when a new file is available in a storage bucket. Use a Pub/Sub-triggered Cloud Function to start the training job on GKE.
 - B. Use Cloud Scheduler to schedule jobs at a regular interval. For the first step of the job, check the time stamp of objects in your Cloud Storage bucket to see if there are no new files since the last run.
 - C. Use App Engine to create a lightweight Python client that continuously polls Cloud Storage for new files. As soon as a file arrives, initiate the Kubeflow Pipelines job on GKE.
 - D. Configure your pipeline with Dataflow, which saves the files in Cloud Storage. After the file is saved, you can start the job on GKE.

The keys are that we want the least effort and the most managed.



Question #26

- You are an MLOps engineer and you deployed a Kubeflow pipeline on Vertex AI pipelines. Which Google Cloud feature will help you track lineage with your Vertex AI pipelines?
 - A. Vertex AI Model Registry
 - B. Vertex AI Artifact Registry
 - C. Vertex AI ML metadata
 - D. Vertex AI Model Monitoring



Question #26

- You are an MLOps engineer and you deployed a Kubeflow pipeline on Vertex AI pipelines. Which Google Cloud feature will help you track lineage with your Vertex AI pipelines?
 - A. Vertex AI Model Registry
 - B. Vertex AI Artifact Registry
 - C. Vertex AI ML metadata
 - D. Vertex AI Model Monitoring

Vertex AI Metadata is used for recording metadata and artifacts produced by the system, as well as the performance of the system and its artifacts. Also use process of elimination here.



Question #27

- What is NOT a recommended way to invoke a Kubeflow pipeline?
 - A. Using Cloud Scheduler
 - B. Responding to an event, using Pub/Sub and Cloud Functions
 - C. Cloud Composer and Cloud Build
 - D. Directly using BigQuery



Question #27

- What is NOT a recommended way to invoke a Kubeflow pipeline?
 - A. Using Cloud Scheduler
 - B. Responding to an event, using Pub/Sub and Cloud Functions
 - C. Cloud Composer and Cloud Build
 - D. Directly using BigQuery

You cannot invoke a Kubeflow pipeline using BigQuery.



Question #28

- You used Vertex AI Workbench notebooks to build a model in TensorFlow. The notebook loads data from Cloud Storage, uses TensorFlow Transform to pre-process data, uses built-in TensorFlow operators to define a sequential Keras model, trains and evaluates the model, and saves the trained model to Cloud Storage for serving. You want to orchestrate the model retraining pipeline to run on a weekly schedule while minimizing running costs, refactoring steps, and monitoring steps. What should you do?
 - A. Add relevant parameters to the notebooks cells and set a recurring run in Vertex AI Workbench.
 - B. Use TensorFlow Extended (TFX) with Google Cloud executors to define your pipeline, and automate the pipeline to run on Cloud Composer.
 - C. Use Kubeflow Pipelines with Google Cloud executors to define your pipeline, and use Cloud Scheduler to automate the pipeline to run on Vertex AI Pipelines.
 - D. Use TensorFlow Extended (TFX) with Google Cloud executors to define your pipeline, and use Cloud Scheduler to automate the pipeline to run on Vertex AI Pipelines.



Question #28

- You used Vertex AI Workbench notebooks to build a model in TensorFlow. The notebook loads data from Cloud Storage, uses TensorFlow Transform to pre-process data, uses built-in TensorFlow operators to define a sequential Keras model, trains and evaluates the model, and saves the trained model to Cloud Storage for serving. You want to orchestrate the model retraining pipeline to run on a weekly schedule while minimizing running costs, refactoring steps, and monitoring steps. What should you do?
 - A. Add relevant parameters to the notebooks cells and set a recurring run in Vertex AI Workbench.
 - B. Use TensorFlow Extended (TFX) with Google Cloud executors to define your pipeline, and automate the pipeline to run on Cloud Composer.
 - C. Use Kubeflow Pipelines with Google Cloud executors to define your pipeline, and use Cloud Scheduler to automate the pipeline to run on Vertex AI Pipelines.
 - D. Use TensorFlow Extended (TFX) with Google Cloud executors to define your pipeline, and use Cloud Scheduler to automate the pipeline to run on Vertex AI Pipelines.

Cloud Composer does not provide ML-specific monitoring. Kubeflow Pipelines does not minimize refactoring steps because the model is in TF. Vertex AI Workbench does not provide alerts, so Option A would require us to log in every week. So we go with the Vertex AI Pipelines in D; only pay for what we use and have lineage capabilities.



Question #29

- You are an ML engineer of a fintech company building a real-time prediction engine that streams files that may contain PII to GCP. You want to use the Cloud Data Loss Prevention API to scan the files. How should you ensure that the PII is not accessible by unauthorized individuals?
 - A. Stream all files to Google Cloud, and then write the data to BigQuery. Periodically conduct a bulk scan of the table using the DLP API.
 - B. Stream all files to Google Cloud, and then write batches of the data to BigQuery. While the data is being written to BigQuery, conduct a bulk scan of the data using the DLP API.
 - C. Create two buckets of data: sensitive and nonsensitive. Write all data to the Nonsensitive bucket. Periodically conduct a bulk scan of that bucket using the DLP API, and move the sensitive data to the Sensitive bucket.
 - D. Periodically conduct a bulk scan of the Google Cloud Storage bucket using the DLP API, and move the data to either the Sensitive or Nonsensitive bucket.



Question #29

- You are an ML engineer of a fintech company building a real-time prediction engine that streams files that may contain PII to GCP. You want to use the Cloud Data Loss Prevention API to scan the files. How should you ensure that the PII is not accessible by unauthorized individuals?
 - A. Stream all files to Google Cloud, and then write the data to BigQuery. Periodically conduct a bulk scan of the table using the DLP API.
 - B. Stream all files to Google Cloud, and then write batches of the data to BigQuery. While the data is being written to BigQuery, conduct a bulk scan of the data using the DLP API.
 - C. Create two buckets of data: sensitive and nonsensitive. Write all data to the Nonsensitive bucket. Periodically conduct a bulk scan of that bucket using the DLP API, and move the sensitive data to the Sensitive bucket.
 - D. Periodically conduct a bulk scan of the Google Cloud Storage bucket using the DLP API, and move the data to either the Sensitive or Nonsensitive bucket.

This represents best practices. You can use Dataflow to write data to BigQuery and then apply the bulk scan using the DLP API.



Question #30

You want to send biweekly scheduled emails to customers that are expected to spend above a variable threshold. After setting up a new Google Cloud project, you use Vertex AI Workbench to develop model training and batch inference with an XGBoost model on the transactional data stored in Cloud Storage. You want to automate the end-to-end pipeline that will securely provide the predictions while minimizing cost and code maintenance. What should you do?

- A. Create a scheduled pipeline on Vertex AI Pipelines that accesses the data from Cloud Storage, uses Vertex AI to perform training and batch prediction, and outputs a file in a Cloud Storage bucket that contains a list of all customer emails and expected spending.
- B. Create a scheduled pipeline on Cloud Composer that accesses the data from Cloud Storage, copies the data to BigQuery, uses BigQuery ML to perform training and batch prediction, and outputs a table in BigQuery with customer emails and expected spending.
- C. Create a scheduled notebook on Vertex AI Workbench that accesses the data from Cloud Storage, performs training and batch prediction on the managed notebook instance, and outputs a file in Cloud Storage that contains a list of all customer emails and expected spending.
- D. Create a scheduled pipeline on Cloud Composer that accesses the data from Cloud Storage, uses Vertex AI to perform training and batch prediction, and sends an email to the marketing team's Gmail group email with an attachment that contains an encrypted list of all customer emails and expected spending.



Question #30

You want to send biweekly scheduled emails to customers that are expected to spend above a variable threshold. After setting up a new Google Cloud project, you use Vertex AI Workbench to develop model training and batch inference with an XGBoost model on the transactional data stored in Cloud Storage. You want to automate the end-to-end pipeline that will securely provide the predictions while minimizing cost and code maintenance. What should you do?

- A. Create a scheduled pipeline on Vertex AI Pipelines that accesses the data from Cloud Storage, uses Vertex AI to perform training and batch prediction, and outputs a file in a Cloud Storage bucket that contains a list of all customer emails and expected spending.
- B. Create a scheduled pipeline on Cloud Composer that accesses the data from Cloud Storage, copies the data to BigQuery, uses BigQuery ML to perform training and batch prediction, and outputs a table in BigQuery with customer emails and expected spending.
- C. Create a scheduled notebook on Vertex AI Workbench that accesses the data from Cloud Storage, performs training and batch prediction on the managed notebook instance, and outputs a file in Cloud Storage that contains a list of all customer emails and expected spending.
- D. Create a scheduled pipeline on Cloud Composer that accesses the data from Cloud Storage, uses Vertex AI to perform training and batch prediction, and sends an email to the marketing team's Gmail group email with an attachment that contains an encrypted list of all customer emails and expected spending.

Cloud Composer is not cost-efficient for one pipeline because its environment is always active. Additionally, entering the Vertex AI Workbench would not minimize code interactions. Lastly, sending private information via email is not recommended.

Conclusion

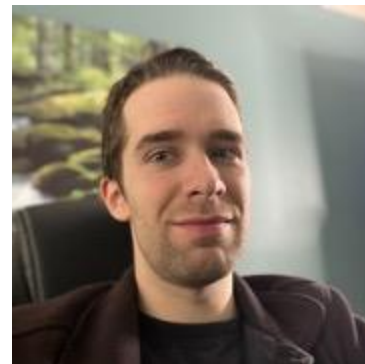
Congratulations! You've reached the end of this crash course.

- If you scored over 60-70% on the practice questions, you are probably in pretty good shape.
- If not, don't worry. Continue reviewing, reading, and practicing on the topics you struggled with.

Connect with me:

- **YouTube:** <https://www.youtube.com/RichardOnData>
- **LinkedIn:** <https://www.linkedin.com/in/ribryan/>

GOOD LUCK!!!



The O'Reilly logo is centered on a blue gradient background. It features the text "O'REILLY" in a white, bold, sans-serif font, followed by a registered trademark symbol (®). The background has a subtle gradient from a darker blue on the left to a lighter blue on the right, with faint, overlapping circular shapes in the lower-left area.

O'REILLY®